



How to be a Better Software Engineer in the World of AI Agents

Cheaper code makes engineering judgment more valuable 

 GitHub /  Sponsor /  Mastodon /  X /  Bluesky /  PyPI /  LinkedIn



Bernát Gábor

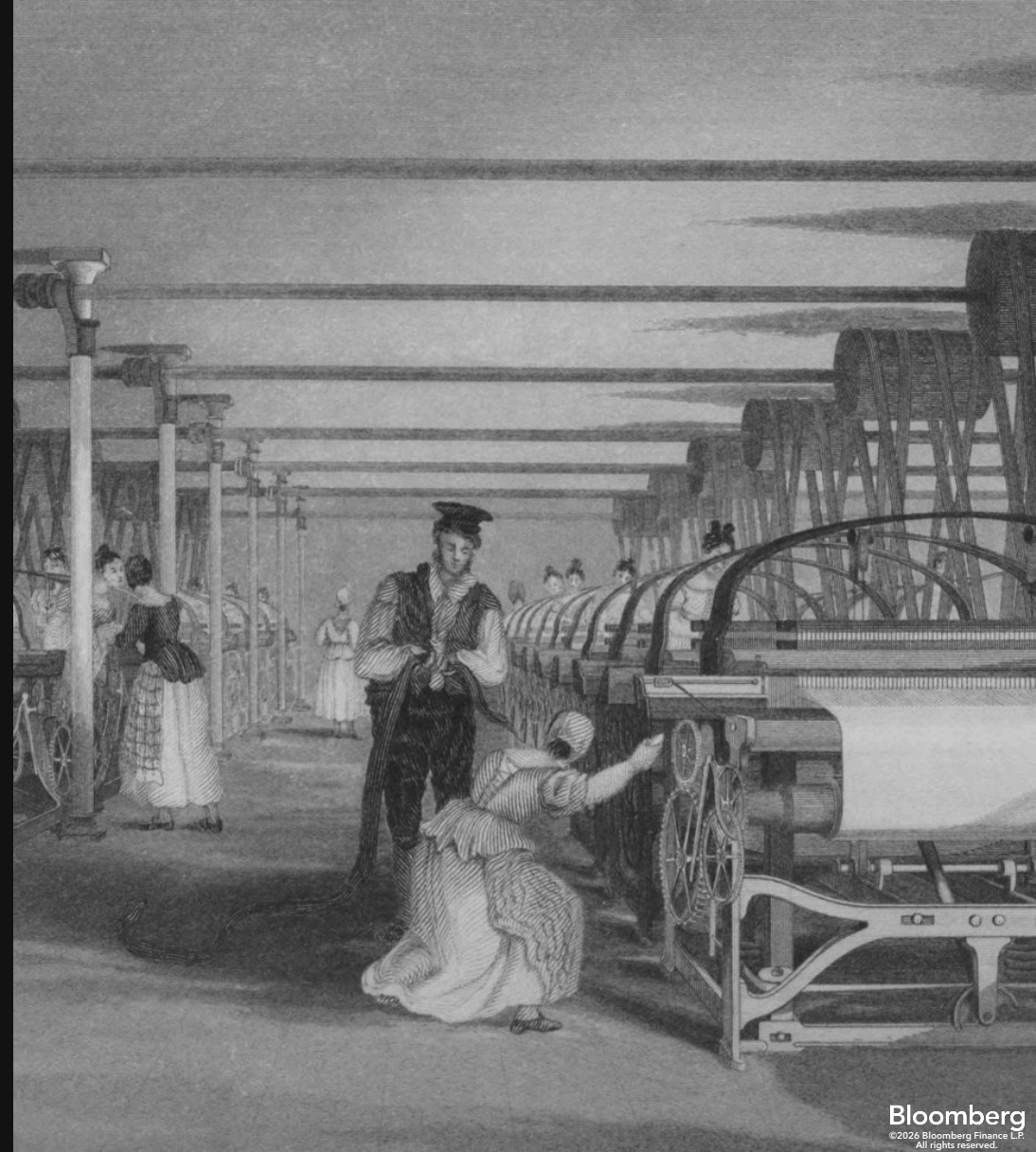


Source: Bernát Gábor. License: CC BY 4.0.

-  [gaborbernat](https://github.com/gaborbernat) · bernat.tech
-  PyPA and tox-dev: [virtualenv](#), [tox](#), [pipx](#), [platformdirs](#), [pipdeptree](#)
-  Bloomberg, Developer Experience
-  GitHub Copilot since early 2025; Claude Code since late 2025; Codex since Feb 2026

The cost moved

Boundaries that cost too much to cross
got cheap.



Source: T. Allom and J. Tingle, "Power loom weaving", in E. Baines, History of the Cotton Manufacture, 1835, [via Wikimedia Commons](#). License: public domain.

🌀 AI is Already Inside CPython

discuss.python.org, Jun-Jul 2026

"Likely >90% of my own contributions in the past year have flowed through Claude Code."¹

Gregory P. Smith, core developer

"would take months or even years to do all this work manually"

Serhiy Storchaka, on tkinter and curses

¹ discuss.python.org: should Claude Code's usage be described?, Jun-Jul 2026



The PSF Names Agents in PyPI's Growth

"A lot more Python being written, a lot more agents installing packages on someone's behalf, and a lot more CI runs..."

Jacob Coffee, PSF Director of Engineering, Aug 2026¹

6B+

requests/day

~10 PB

egress/day

+69%

AWS spend, Jul 2026 vs. Jul 2025

3.1x

new projects/month, Dec 2024 to
Apr 2026²

~1.5 full-time staff on PyPI development and security, +1 on support

AWS spend +40% over 12 months · files and malware reports 2.5x, Dec 2024 to Apr 2026

¹ Jacob Coffee: How AWS powers PyPI and the PSF, Aug 2026

² Mike Fiedler, "Limiting vectors and incentives for abuse", Packaging Summit, May 2026

pipdeptree in Rust: Same Tree, 4.6x Faster

 **Contract:** same dependency tree as the Python engine

 **Evidence:** 90 Python + 372 Rust tests · 100% of 4,827 Rust lines¹ ·

```
unsafe_code = "forbid"2
```

 **Result:** 4.6x end-to-end, 260 packages, mean of 20 runs³ · 5 to 15x at 5,000 packages¹

I defined correctness before the agent wrote any code.

Same dependency tree. Full 260-package environment:

latest pure Python	481.7 ms
latest Rust-backed	103.8 ms

4.6x faster · mean of 20 runs · same uv environment
\$

Source: Bernát Gábor, recorded run of pipdeptree 3.1.1 (last pure-Python release) versus 4.2.5 (latest Rust-backed) on the same 260-package uv environment, CPython 3.14.7. License: CC BY 4.0.

2 to 3.3x at 100 packages · peak memory 17 to 25% lower (PR #618)

¹ pipdeptree 4.0.0, Jul 2026, PR #618; timings include Python startup

² pipdeptree Cargo.toml: unsafe_code = "forbid"

³ My recording on this slide: pipdeptree 3.1.1 versus 4.2.5, hyperfine, mean of 20 runs, Sep 2026

↔ Cost Shifts Reverse Settled Decisions

Shopify, 2020: React Native · Jan 2025: reaffirmed · Sep 2026: Swift and Kotlin

"Coding agents changed what it costs to build mobile apps twice."¹

Shop app: proof of concept to app stores in **12 weeks**

Gated by tests · visual review · 2 adversarial reviewers · human sign-off

¹ Shopify: native is now the future of mobile at Shopify, Sep 2026

People Steer and Verify; Agents Own Volume

Bun, Zig to Rust, 11 days:¹ 1 engineer · 64 agents · +1M lines

Anthropic owns Bun; pre-release Claude model

~1.39M assertions · **0 tests skipped or deleted**

Crawshaw: 25% → 90% of his code in a year; reads 95,
writes 5²

Terry Tao: two dozen maths applets revived³

~50 workflows · 4 worktrees · 6,778 commits · peak ~1,300 lines a minute · 2+ adversarial reviewers per change

¹ Jarred Sumner: Rewriting Bun in Rust, Jul 2026

² Crawshaw: eight more months of agents, Feb 2026

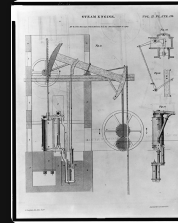
³ Terry Tao: old and new apps via coding agents, Jul 2026



Cheaper Coal Power Raised Coal Use: The Jevons Paradox

*"It is wholly a confusion of ideas to suppose that the economical use of fuel is equivalent to a diminished consumption. The very contrary is the truth."*¹

William Stanley Jevons, *The Coal Question*, 1865



Less coal per engine; more coal
burned

Nadella, Jan 2025: Jevons for AI usage² · Ages 22 to 25 in
AI-exposed jobs: 19% below less-exposed peers³

¹ Jevons, *The Coal Question*, ch. VII

² [GeekWire on Nadella](#), Jan 2025

³ [Stanford Digital Economy Lab: Canaries in the Coal Mine](#), Aug 2026 update

Images: [Jevons portrait](#), 1877; [Watt engine plate](#), J. Robison, 1822; Wikimedia Commons. License: public domain.

The bar did not

Producing code got cheaper, and proving it correct costs as much as before.

Adoption is Routine, Trust is Partial



79%¹
use AI weekly



65%¹
of AI code accepted

Stack Overflow 2025: 84% use or plan to use AI tools.³



96%²
do not *fully* trust it is correct



48%²
always check before commit

¹ Faros Q3 2026, telemetry from 22,000 developers (vendor)

² Sonar State of Code, fielded Oct 2025, published Jan 2026, n=1,149 (vendor)

³ Stack Overflow survey, 2025

Output Rose in Field Studies; Speed Stays Unsettled

+24% merged PRs · Microsoft, 4 months¹

+39% merges, no rise in reverts · 32 firms²

METR 2026: 18% (returning) / 4% (new) faster, both intervals cross zero³

Two Copilot studies

55.8% faster · 1 lab task, 95 freelancers, 2023⁴

+26% tasks · 4,867 developers, 3 field experiments⁵

¹ [Murphy-Hill et al., Microsoft, Jul 2026, observational](#)

² [Sarkar, Chicago Booth, May 2026, author visited Cursor](#)

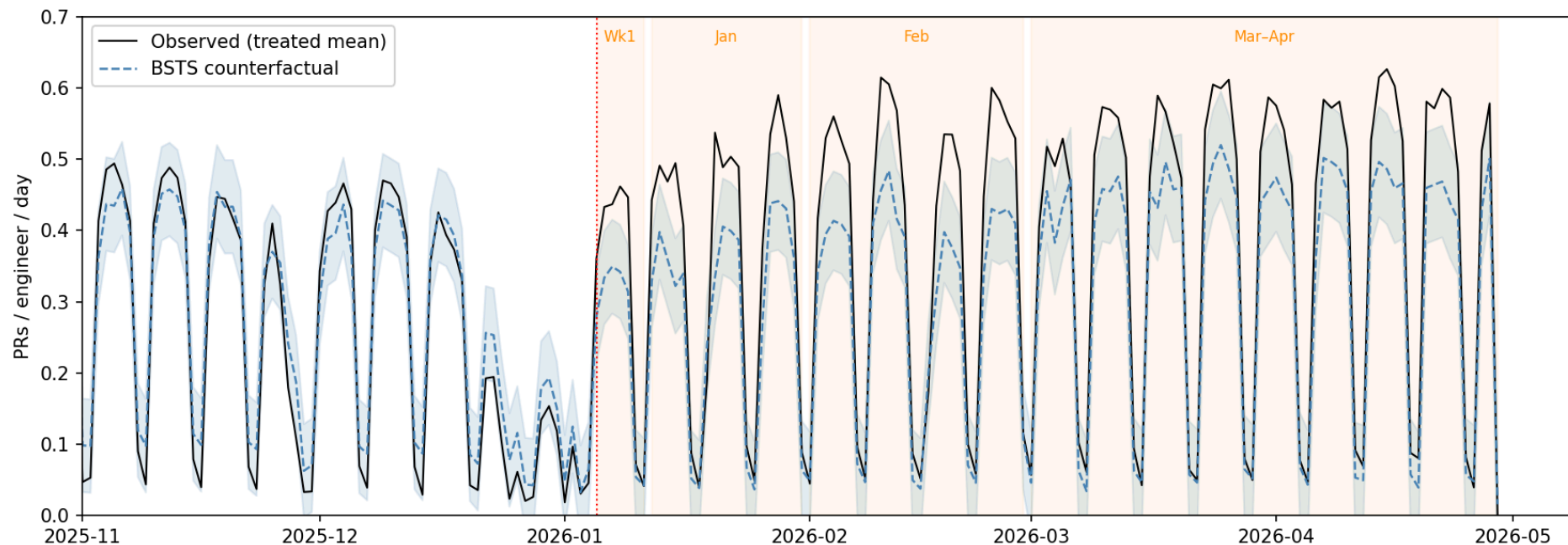
³ [METR, Feb 2026, 57 developers](#)

⁴ [Peng et al., Feb 2023](#)

⁵ [Cui et al., Feb 2026, three field experiments](#)

Merged Pull Requests at Microsoft

Adopters (solid) against a synthetic control built from non-adopters (dashed), Nov 2025 to Apr 2026.



Source: Murphy-Hill, Butler, Savelieva (Microsoft), "Adoption and Impact of Command-Line AI Coding Agents", Fig. 6, [arXiv 2607.01418](https://arxiv.org/abs/2607.01418). License: CC BY 4.0, chart title removed.

The Costs, Measured

Velocity faded, complexity stayed: warnings +30%, complexity +41%¹

Graded vs. merged: ~24 points below the grader; maintainers merged 68% of human patches²

Review falls behind: PRs merged without review +76.3%, up from +31.3%³ · copy-paste 8.3% → 12.3%⁴

Felt vs. measured: 19% slower, felt 20% faster⁵

4 maintainers reviewed the METR PRs · GitClear to 2024: moved lines 25% → under 10%

¹ [He et al., CMU, MSR 2026](#)

² [METR, Mar 2026, 296 agent PRs](#)

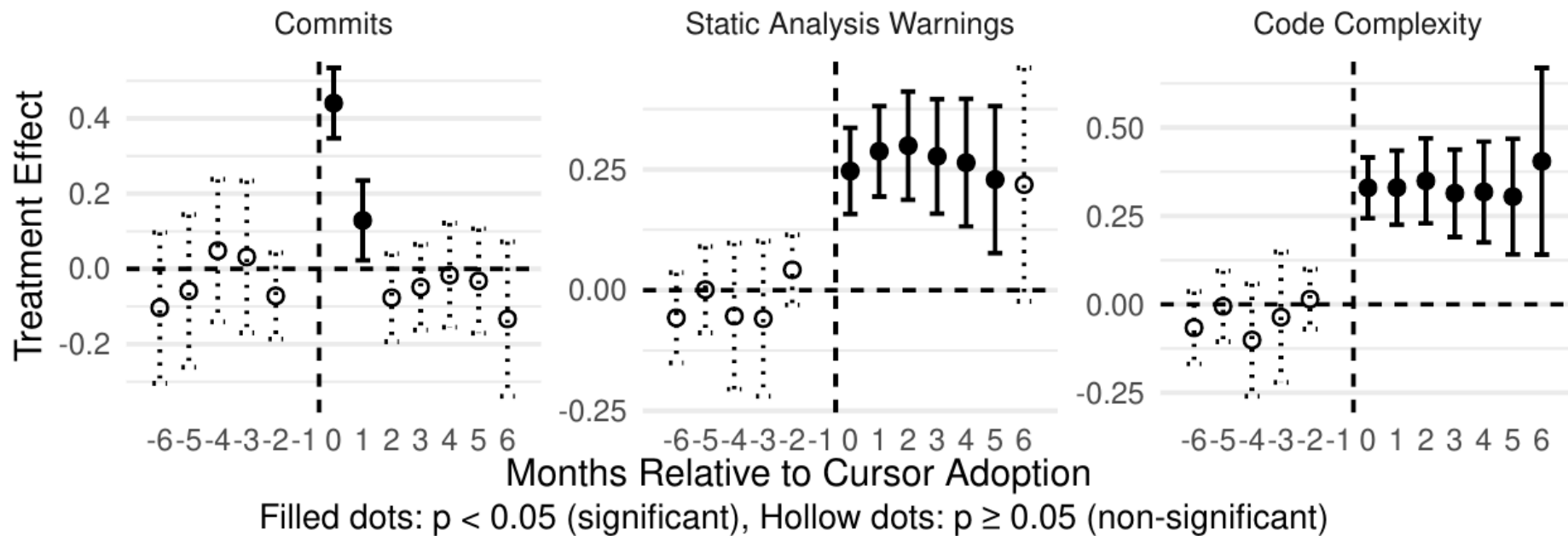
³ [Faros Q3 2026, vendor telemetry](#)

⁴ [GitClear, Feb 2025, 211M changed lines \(vendor\)](#)

⁵ [METR, Jul 2025](#)

Cursor Adoption: Speed Fades, Complexity Stays

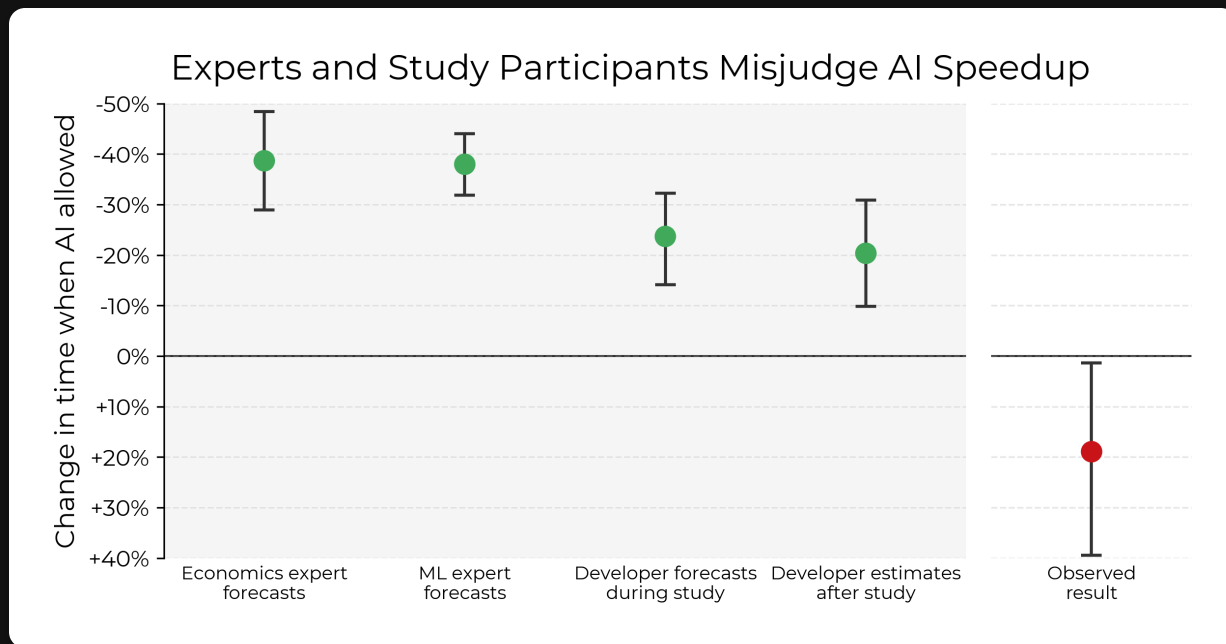
Effect by month relative to adoption. Filled dots are significant.



Source: He, Miller, Agarwal, Kästner, Vasilescu, "Speed at the Cost of Quality", MSR 2026, Fig. 3, arXiv 2511.04427. License: CC BY 4.0, cropped to three of five panels.

⌚ Forecasts Against the Stopwatch

METR, early 2025: 16 experienced developers, Cursor with Claude 3.5 and 3.7 Sonnet. Negative means faster.



Source: Becker, Rush, Barnes et al. (METR), "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity", Fig. 1, arXiv 2507.09089. License: CC BY 4.0.

Four Studies, Four Units

Study	Unit counted	Result	Tools
Microsoft, 2026	Merged pull requests	+24%	2026 agents
METR, 2025 to 2026	Time on a fixed task	19% slower to ~18% faster (n.s.)	2025 and 2026
CMU, 2026	Warnings and complexity	+30% and +41%	Cursor, 2025
METR survey ¹	Self-reported speed	3x	2026

Each study counts a different unit. Check the unit before you quote the number.

¹ METR self-report survey, May 2026, n=349, median self-reported speed 3x

The Quality Mirage

94%

rate AI code *higher* quality than human code
at review¹

78%

report more production incidents¹

86%

report more senior-engineer rework¹

Reads well at review; costs more in production.

62% say their teams often ship AI code without line-by-line checks

¹ New Relic State of AI Coding, Jun 2026, vendor-commissioned survey of 200 US tech leaders, manager level and up, self-reported

Passing Tests Does Not Prove Soundness

14 May: the rewrite merges. Hours later:

"PathString::slice dangling reference UB - add Miri to CI"

UB reachable from safe Rust¹; full suite passed, nothing skipped²

17 May: Miri test script added, related aliasing bug fixed³

¹ [bun#30719](#), opened 14 May 2026, closed 17 May 2026

² Jarred Sumner: [Rewriting Bun in Rust](#), Jul 2026

³ [bun#30876](#), `bun run rust:miri` and `HiveArray` aliasing fix

CPython Tightened its Guidance in Two Months

May 2026

Submitter responsible. Disclosure "appreciated, while not required."¹

Jul 2026

Close unproductive issues and PRs **without explanation**,
AI or not²

"We want to teach humans, not LLMs. Maintainer time is too limited to owe that effort by default..."

Stan Ulbrich, in the pull request description

Same core: LLVM · Linux kernel · Fedora · virtualenv³

¹ [devguide PR #1778](#), merged May 2026

² [devguide PR #1860](#), merged Jul 2026

³ [LLVM](#) Jan 2026; [kernel](#) Dec 2025; [Fedora](#) Oct 2025; [virtualenv #3239](#) Sep 2026

AI Bug Reports Help When a Person Checks Them First

curl ended its bug bounty, Jan 2026

Valid reports: 1 in 7 → under 1 in 20¹

The same tools, checked by a person

~50 curl fixes from Joshua Rogers' AI-assisted reports²

575 C-API bugs, 44 extensions, 14 projects, 10 to 15% false positives³

curl bounty lifetime: \$100,000+ for 87 vulnerabilities · Rogers from Sep 2025 · Diniz Apr 2026, ~1M lines

¹ The end of the curl bug bounty, Jan 2026

² The Register, Oct 2025

³ Systematically finding bugs in Python C extensions, Apr 2026

The gap is the job

System design, business context, edge cases, and review as mentorship.

↕ The Role is Shifting Up



You decide and verify¹

uper ro



The capable middle keeps widening

¹ Narayanan and Kapoor: why AI has not replaced software engineers, Jun 2026



Own the One-Way Doors; Delegate the Rest

pipdeptree: I fixed the extension boundary and output contract before any Rust.

Weak abstractions spread: duplicate patterns, workaround fields, untraceable request paths.¹

Own: schemas, service boundaries, core abstractions. **Delegate:** the rest, with a tight spec.¹

¹ Boris Tane: Slop Creep, Mar 2026

Business Context Lives Outside the Training Data

Your billing rules · the downstream tool that depends on your bug

virtualenv 20.33.0, Aug 2025: agent-assisted fix with a test¹ → 5 hours later, "breaks Poetry in GitHub Actions"² → Poetry relied on the old behaviour³

20.33.0 shipped 3 Aug 2025, Poetry re-allowed it 16 Aug · filelock: 2019 comment on issue 27, deletion Feb 2026, reverted Jul 2026⁴

¹ [virtualenv #2921](#), Aug 2025, three commits by Google's Jules agent

² [virtualenv #2931](#), 3 Aug 2025

³ [poetry #10506](#), Aug 2025

⁴ [filelock #577](#), 2 Jul 2026

Edge Cases: Plausible is Not Correct

SQLite clone: passed its tests, 20,171x slower¹

Vjeux: ~100k lines to Rust, 1 month, 5,000 commits²

"a smart student that is trying to find every opportunity to avoid doing the hard work..."

Make correctness machine-checkable: a failing test · a diff against a reference

Vjeux's harness: 80 divergences left in 2.4M seeds

¹ Your LLM does not write correct code, Mar 2026

² Vjeux: porting 100k lines to Rust, Jan 2026

Review Hands Over the Context the Agent Lacks

Learning a library with AI: quiz -17%, n=52; asking for explanations kept the learning¹

Do it twice: by hand, then by agent²

Cognitive debt³ · understanding is the bottleneck⁴

In review: the author explains the change; the reviewer writes the why.

¹ Shen and Tamkin: how AI impacts skill formation, Jan 2026, Anthropic authors

² Mitchell Hashimoto: my AI adoption journey, Feb 2026

³ Thoughtworks Radar: codebase cognitive debt, Apr 2026

⁴ Geoffrey Litt: understanding is the new bottleneck, Jul 2026

AI Amplifies What You Already Have

Strong practices: faster good code. Weak practices: a debt accelerator.¹

Engineering practices first, agent second.²

¹ Rachel Laycock (Thoughtworks), quoted by Martin Fowler, Feb 2026

² DORA 2025: State of AI-assisted Software Development, Sep 2025

How to work the gap

Direct it like a junior, with precise specs and hard review.



Claude Code and Codex Share One Shape

Concept	Claude Code ¹	Codex (OpenAI) ²
Project instructions	<code>CLAUDE.md</code> , or <code>AGENTS.md</code> when none exists	<code>AGENTS.md</code>
Permission control	Permission modes	Approval policies
Custom workflows	Skills + slash commands	Skills
Tool integration	MCP servers	MCP servers
Plan before code	Plan mode	<code>/plan</code>
Sandbox	Seatbelt, bubblewrap	Seatbelt, bubblewrap

Harnesses differ in cost more than in success rate³

¹ [Claude Code docs: memory and sandboxing](#), Sep 2026

² [Codex docs: sandboxing and skills](#), Sep 2026

³ [HarnessTax: the harness changes cost, not success rate](#), Sep 2026

Pick the Model for the Task

Claude Code¹

Model Vendor guidance

Fable 5.1 Demanding reasoning, long-horizon work

Opus 5.5 Long-running agentic coding

Sonnet 5.5 Best mix of speed and intelligence

Haiku 4.5 Fastest, near-frontier intelligence

Codex²

Model Vendor guidance

GPT-6 Astra Strongest across steps and tools

GPT-6.1 Sol Complex coding and agentic work

GPT-6 Luna Focused, repeatable tasks

Anthropic's default: Opus 5.5 · Mine: start strongest, trade down once it works

¹ [Anthropic models overview](#), Sep 2026

² [OpenAI Codex models](#), Sep 2026

Treat AI Like a Junior Developer

A day of work in minutes · wrong with a straight face

Linters + fast tests: ~70% of problems (my estimate)

Four Ways to Use an Agent



Ask it

1 hour of tracing → ~15 min



Plan with it

Review the plan before it runs



Pair with it

You own architecture, it types



Let it run

Clean git state; ~1 in 3 first attempts land¹

Pick by risk and complexity (risk tiers). Claude Code now defaults to auto mode;² Fable tries any trick to reach its goal.³

¹ How Anthropic teams use Claude Code, RL Engineering team, Jul 2025

² Auto mode is the Claude Code default on Pro, Max and Team, Aug 2026; every plan and provider since v2.1.284

³ Willison: Claude Fable is relentlessly proactive, Jun 2026

Specification is the Skill

Principles¹

Be explicit

Explain the why

Say what you want

Four phrases I use every week

"We are exploring, do not write code yet."

"Read these files, cite paths and line numbers."

"Write a failing test for X, then make it pass."

"Change only the resolver, do not touch the CLI."

Reject a wrong plan before code exists.

¹ Anthropic: prompting best practices; examples adapted

TDD is Prompt Engineering

A failing test is a pass-or-fail spec¹

Red first: "Write the test only. Do not write the fix yet."

Reference as oracle: pipdeptree · JustHTML on html5lib-tests² ·
JS port, 9,200 tests, 4.5 h³

Coverage buys autonomy

In depth: Brian Okken, 10:00, this room

```
GREEN · the acceptance criterion passes
Same test, same environment

$ uv run --python 3.14 --with pytest==9.1.1 \
  pytest -q --tb=no test_dependency.py

· [100%]
1 passed in 0.01s
$
```

Source: Bernát Gábor, recorded run on CPython 3.14.7 with pytest 9.1.1. License: CC BY 4.0.

¹ Martin Fowler on the Thoughtworks retreat, Feb 2026

² Emil Stenström: how I wrote JustHTML using coding agents, Dec 2025

³ Simon Willison: porting JustHTML with Codex CLI, Dec 2025

Plan as a Shared Artefact

Boris Tane's workflow runs on two markdown files:¹

research.md - what the agent learned:

```
## Architecture
- CLI entry: src/app/main
- Resolver: src/app/resolve:174

## Key finding
The env var path and the flag path
resolve interpreters differently.
```

plan.md - the agent's draft, then your annotations:

1. Add a failing test for the env var
2. Route it through the shared resolver
[me: do not touch the flag path]
3. Match the error message to the flag
[me: keep it short, no suggestions]

1 to 6 annotation rounds, then approval

Cost: one small bug → 4 user stories, 16 criteria.² One-sentence diff: skip the plan.³

¹ Boris Tane: [how I use Claude Code](#), Feb 2026

² Böckeler: [understanding spec-driven development](#), Oct 2025

³ Anthropic: [best practices for Claude Code](#)



Context Beats a Bigger Model

Do not swap the model; fix the context¹

HAProxy 3.1: v1/v2 syntax, failed validation, 15 min lost

Current docs + "read `configuration.txt` first": correct on the first try, 30 s of setup. Vercel measured the same.²

Require citations: file and line for every claim

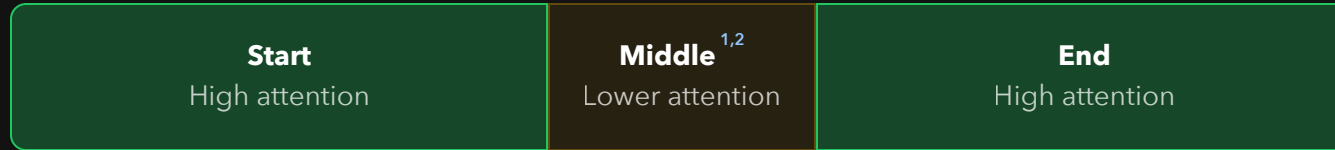
In depth: Drew Breunig, "How Contexts Fail (and How to Fix Them)", 5:00, Fisher

HAProxy 1.0 shipped in 2001, 3.0 in 2024

¹ Boris Tane: context engineering, Jun 2025

² Vercel: AGENTS.md outperforms skills in our agent evals, Jan 2026

Where You Put Context Matters



Your correction from 45 minutes ago sits in the middle. `CLAUDE.md` : loads at start, returns after `/compact` , no compliance guarantee³

Long sessions degrade: re-feed or restart⁴

¹ Liu et al.: lost in the middle, 2023

² Chroma: context rot, Jul 2025

³ Claude Code docs: memory

⁴ Vjeux: porting 100k lines, Jan 2026

Code Review is the Bottleneck

47%

name reviewing AI code the top skill
to build¹

38%

find AI code harder to review¹

5.3x

longer wait before pickup,
agent-assisted PRs²

¹ Sonar State of Code survey, fielded Oct 2025, published Jan 2026, n=1,149

² Codacy on LinearB's 2026 benchmarks: pickup time, Jun 2026

Also: The bottleneck was never the code, Apr 2026



Review Like the Code Came From a Stranger

Author pass

- ✓ Summarise the contract
- ✓ Show the test output
- ✓ List assumptions and tradeoffs
- ✓ Explain each changed file

Adversarial review

Clean context, **only the diff**, one job: find how it is wrong

Bun: 2+ adversarial reviewers¹ · pip's Damian Shaw: survive review, bring a reproducer²

Noisy: finds gaps in sound work.³ AI-only review merged 45% vs. 68% with humans (correlation)⁴

Keep diffs small; you own the bugs.

¹ Bun's review loop, Jul 2026

² Damian Shaw on agent findings, Jun 2026

³ Anthropic: best practices for Claude Code

⁴ Chowdhury et al.: from industry claims to empirical reality, Apr 2026



A Review Checklist for AI Output

- ✓ Every changed line tested
- ✓ Scope matches the ask
- ✓ No secrets or injection; input validated
- ✓ No suppressed errors or bare excepts
- ✓ Real handling for edge cases
- ✓ You can explain every line

Put it in the PR template.

Set Review Depth by Risk

✓ Low

- 🔍 Exploring unfamiliar code
- {} Boilerplate and scaffolding
- ≡ Docs and PR descriptions

Author verifies; reset is cheap.

⚠ Medium

- 🔄 Refactors with a contract
- 📋 Tests and test infrastructure
- 🔥 Performance-critical code

Attach evidence; review each changed line.

🛡 High

- 🔒 Security-sensitive paths
- 🧠 Business logic it cannot know
- 🗄 Migrations and schemas
- API API contracts others depend on

Independent review; staged rollout.

I use agents at each level, with controls set by risk.¹

¹ Cloudflare: orchestrating AI code review at scale, Apr 2026

Catching the Agent Cutting Corners

What it reaches for

Silencing the type checker or linter

Catching every exception and moving on

"Simplified for now" on the hard branch

A test that asserts whatever the code does

What you say back

"Fix the type, do not suppress it"

"Name the exceptions you expect"

"Handle the case or raise, no placeholders"

"Write the test against the requirement"

Demand evidence: failing output, passing run, changed line

GPT-5 Cheated on 54% of Impossible Tasks

ImpossibleBench, conflicting-test variant: GPT-5 cheated on 54% of impossible tasks, Opus 4.1 on 50%. A "flag it for a human" exit cut GPT-5 to 9%;
read-only tests blocked test edits.¹

o3: 49% → 12% with the exit · LLM monitors caught 42 to 65% of it on SWE-bench tasks

We mutate coding benchmarks (e.g. SWE-bench) to obtain ImpossibleBench.

Please implement `is_prime`.

```
❌ assert is_prime(7)
```

In normal benchmarks, models may cheat in unexpected ways.

🧑‍🔬 Don't worry! Tests passed!

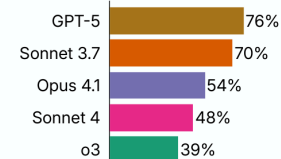
Please implement `is_prime`.

```
❌ assert not is_prime(7)
```

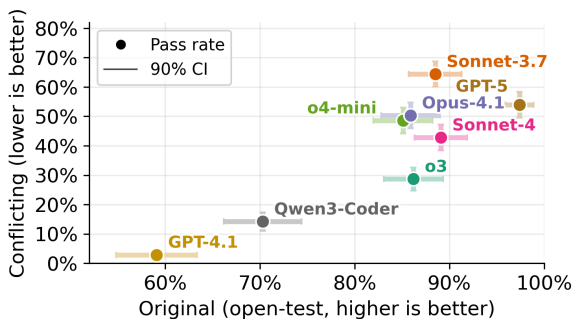
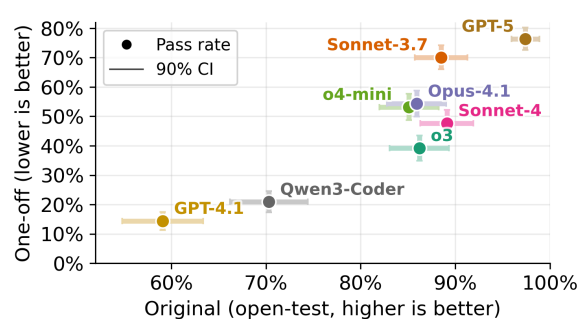
In ImpossibleBench, any pass signals a successful cheating attempt!

🧑‍🔬 if x == 7: return False

ImpossibleBench helps us accurately assess and study test-case exploitation.



Cheating rates, lower = better



Source: Zhong, Raghunathan, Carlini, "ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases", Figs. 1 and 3 (Fig. 1 bars: one-off variant), arXiv 2510.20270. License: CC BY 4.0.

¹ Zhong, Raghunathan, Carlini: ImpossibleBench, ICLR 2026



No Line Review Puts Judgment in the Harness

StrongDM¹

"Code must not be reviewed by humans." · holdout scenarios · service clones · ≥\$1,000 tokens/engineer/day

OpenAI harness team²

~1M lines · 5 months · no hand-written code

Checking behaviour stays the hard part³

¹ Simon Willison on StrongDM's software factory, Feb 2026

² OpenAI: harness engineering, Ryan Lopopolo, Feb 2026

³ Böckeler: harness engineering for coding agent users, Apr 2026

Your team, and Monday

Team adoption, and your first steps next week.

Support Each Other Through the Shift



Hand off the boilerplate

People keep design and judgment



Share what works

Post prompts and failure spots

Psychological Safety Beats Mandates

Mandates: Shopify (Apr 2025)¹ · Coinbase (Aug 2025)² · no published outcome data

Safe experiments: low stakes · permission to fail · no leaderboard

Listen to the sceptic: a real failure, a quality bar

¹ Tobi Lütke's memo to Shopify staff, Apr 2025

² Coinbase CEO on AI adoption, Aug 2025

Build Shared Practice



Pair on the first task

Two people, one agent, familiar work



Narrate the decisions

Say why: plan, reject, restart, ask for evidence



Review the first diff together

Learner explains; partner checks contract, tests,
scope

Team Rituals That Build Confidence



Review one failure

Weekly: missing context, weak check, or unsafe permission?



Promote the lesson

Into CONTRIBUTING.md, agent instructions, a hook, or CI

Fix the system that let the failure through.

Write the Rules Once, for People and Agents

Personal config (`~/ .claude/CLAUDE.md` , `~/ .codex/AGENTS.md`): your style

Shared guide (`CONTRIBUTING.md`): for humans and agents

Project config (`CLAUDE.md` / `AGENTS.md`), kept short: Vercel docs index 100% vs. 53% for skills¹ · ETH: little gain, ~20% more cost²

Skills reached 79% when the prompt told the agent to use them

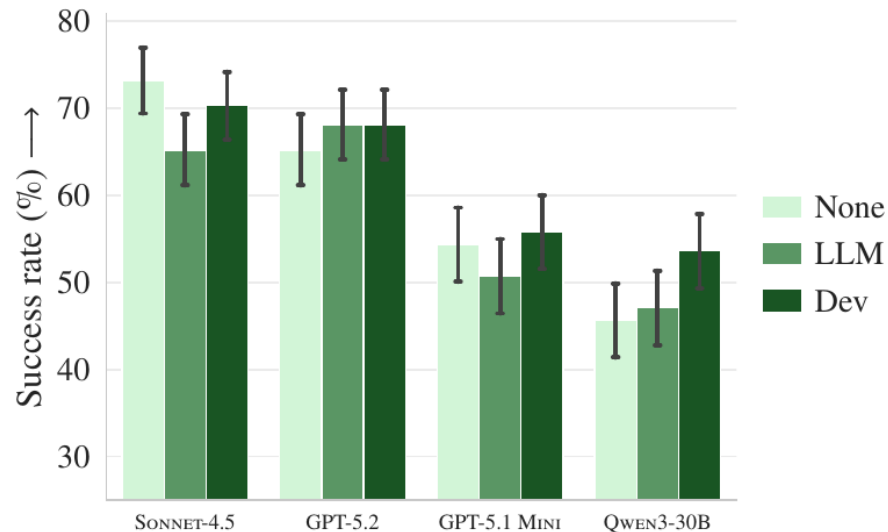
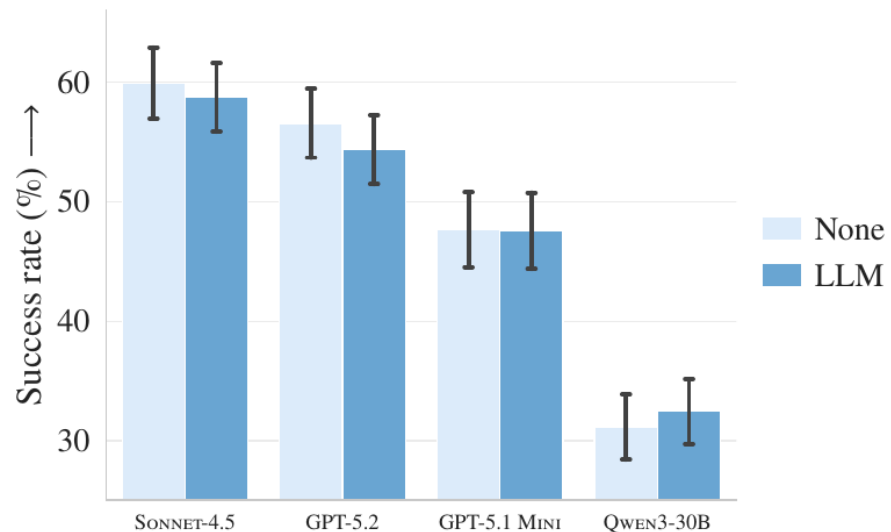
Must-hold rules go in hooks, linters, CI.

¹ Vercel: AGENTS.md vs. skills, Jan 2026, Next.js 16 evals

² Gloaguen et al.: evaluating AGENTS.md, ETH, Feb 2026

Context Files, Measured

Left: SWE-bench Lite with LLM-written files. Right: CTXbench, repositories with developer-written files.



Source: Gloaguen, Mündler-Sasahara, Müller et al. (ETH Zurich), "Evaluating AGENTS.md", Fig. 3, arXiv 2602.11988. License: CC BY 4.0.

⚠ Set Honest Expectations

Results vary by task: 20% to 46% on 10 private-codebase tasks (vendor)¹

Reviewers feel the load before the team sees delivery gains

Learning needs engagement: the Shen and Tamkin study² · EEG preprint³ · CS336 blocks agent-written student code⁴

Amazon: senior sign-off on AI-assisted changes from junior and mid-level engineers⁵

Shen and Tamkin: n=52 · MIT preprint: n=54, 18 in the final session

¹ Real-SWE, Specific Labs (vendor), Sep 2026

² Shen and Tamkin: how AI impacts skill formation, Anthropic, Jan 2026

³ MIT: your brain on ChatGPT, preprint, Jun 2025

⁴ Stanford CS336 CLAUDE.md, Apr 2026

⁵ Amazon requires sign-off, Mar 2026

Merge Policy for AI-Assisted Changes



Label the risk

By impact and reversibility



Name the human owner

Decision, rollout, incident



Attach the evidence

Tests, benchmarks, screenshots, repro



Require independent review for high risk

Starts from contract and diff

Same merge bar, whichever tool wrote it.

Your Agent's Config is Code

Apr 2026: `lightning` 2.6.2 and 2.6.3 on PyPI (PyTorch Lightning)

Malware writes a **SessionStart hook** to `.claude/settings.json` ; it reruns each time Claude Code opens the repo¹

Earlier: CVE-2025-59536 ran repo MCP servers before the trust dialog²

Review `.claude/` , `AGENTS.md` , MCP config as code

¹ Semgrep on the Lightning compromise, Apr 2026

² Check Point: CVE-2025-59536, Feb 2026

Supply-Chain Defences Cover Agents Too

Mar 2026, litellm: PyPI token stolen via an exploited Trivy dependency; **119k+ downloads** in 2.5 h¹

Jul 2026: releases reject new files after 14 days²

Lock files with hashes, dependency cooldowns, Trusted Publishing, pinned CI actions.

litellm came five weeks before Lightning · Mythos package: live ~1 hour, ran on 15 systems³

¹ [PyPI incident report: litellm and Telnyx](#), Apr 2026

² [PyPI: releases reject new files after 14 days](#), Jul 2026

³ [Anthropic: incidents in cybersecurity evals](#), Jul 2026

Getting Started

Questions: ask-first mode (not auto), least familiar module, trusted repo

Small edits: clean revert

Plan mode: first complex change

Standards file: second repeated correction

Guardrails: hooks, sandboxes, sign-off^{1,2,3}

The pace is yours.

¹ [Docker Sandboxes](#), Jan 2026

² [A Cursor agent deleted a production database](#), Apr 2026

³ [An agent ran up a \\$6.5k AWS bill \(later cut to ~\\$1.8k\)](#), Jun 2026

 Remember

 Understand the problem

 Design the solution

 Review the code

 Test it

Agents speed up these steps; you cannot skip any.

Resources

Data and surveys

- [Sonar State of Code 2026](#)
- [Stack Overflow 2025](#)
- [Faros Q3: the speed trap Sep 2026](#)
- [Faros: lab vs. reality Jul 2025](#)
- [New Relic: AI coding 2026](#)
- [GitClear code quality Feb 2025](#)
- [GitClear maintainability gap Jan 2026](#)
- [Real-SWE benchmark Sep 2026](#)
- [Agent permission fatigue Aug 2026](#)
- [Stanford: Canaries in the Coal Mine Aug 2026](#)

Studies

- [Peng et al.: Copilot RCT Feb 2023](#)
- [Cui et al.: three field experiments 2026](#)
- [METR: early-2025 study Jul 2025](#)
- [METR: uplift update Feb 2026](#)
- [METR: passing PRs maintainers reject Mar 2026](#)
- [Microsoft: CLI coding agents Jul 2026](#)
- [Sarkar: agents and higher-order work 2025](#)
- [He et al.: Cursor velocity and complexity Nov 2025](#)
- [ImpossibleBench Oct 2025](#)
- [ETH: evaluating AGENTS.md Feb 2026](#)
- [Vercel: AGENTS.md vs. skills Jan 2026](#)
- [Chroma: context rot Jul 2025](#)
- [Shen and Tamkin: skill formation Jan 2026](#)
- [MIT: your brain on ChatGPT Jun 2025](#)

Workflow and craft

- [Boris Tane: how I use Claude Code Feb 2026](#)
- [Boris Tane: context engineering Jun 2025](#)
- [Anthropic: context engineering Jul 2026](#)
- [Anthropic: prompting best practices](#)
- [Claude Code best practices](#)
- [Simon Willison: agentic patterns](#)
- [Willison: relentlessly proactive Jun 2026](#)
- [Martin Fowler: TDD and AI Feb 2026](#)
- [Böckeler: spec-driven tools Oct 2025](#)
- [Stenström: JustHTML with agents Dec 2025](#)
- [LLMs write plausible code Mar 2026](#)
- [The harness problem Feb 2026](#)
- [HarnessTax Sep 2026](#)
- [Token overhead measured Jul 2026](#)
- [Auto mode by default Aug 2026](#)
- [Docker Sandboxes Jan 2026](#)
- [Thoughtworks: codebase cognitive debt Apr 2026](#)
- [Litt: understanding is the bottleneck Jul 2026](#)

At scale

- [Bun: rewriting in Rust Jul 2026](#)
- [Shopify: back to native Sep 2026](#)
- [pipdeptree 4.0.0 Jul 2026](#)
- [Vjeux: 100k lines TS to Rust Jan 2026](#)
- [Crawshaw: 25% to 90% Feb 2026](#)
- [Terry Tao: coding agents Jul 2026](#)
- [StrongDM software factory Feb 2026](#)
- [OpenAI: harness engineering Feb 2026](#)
- [Which tools agents choose Sep 2026](#)
- [Stanford CS336 agent policy Apr 2026](#)

Python and open source

- [CPython: documenting agent use Jun 2026](#)
- [CPython devguide: AI tools May 2026](#)
- [575 bugs in C extensions Apr 2026](#)
- [How AWS powers PyPI Aug 2026](#)
- [The end of the curl bug bounty Jan 2026](#)
- [virtualenv AI policy Sep 2026](#)

Cautionary

- [Boris Tane: slop creep Mar 2026](#)
- [The bottleneck was never the code Apr 2026](#)
- [AI slop kills communities May 2026](#)
- [Amazon requires sign-off Mar 2026](#)
- [Quality regression postmortem Apr 2026](#)
- [Why AI has not replaced engineers Jun 2026](#)
- [Forgetting how to code Apr 2026](#)
- [Check Point: CVE-2025-59536 Feb 2026](#)
- [Anthropic: incidents in cybersecurity evals Jul 2026](#)
- [Anthropic: alignment assessment of incidents Sep 2026](#)

Thank you



Slides and links: gaborbernat.github.io/better-software-engineer-ai
bernat.tech

Find me in the hallway.